

AI Stack Ecosystem

Setup Guide and Reference

Dual-Stack Architecture: Free Unlimited + Paid Limited

Compatibility Matrix, Contradictions, Fallbacks, and Recommendations

Installer Script v2.0 with Audit Fixes

June 2026 | Ubuntu 24.04 LTS | 8GB RAM

Audit Fixes Applied: Single-hop billing proxy, health-check middleware, consolidated OAuth secrets, unified config paths, version-pinned binaries, Prometheus metrics, Redis TTL bloom dedup, integration tests.

Table of Contents

1. Executive Summary
2. AI Stack Ecosystem Overview
3. Tool Deep Dive: Free Unlimited Stack
4. Tool Deep Dive: Paid Limited Stack
5. Tool Deep Dive: Hybrid and Optional Tools
6. Compatibility Matrix
7. Contradictions and Conflicts
8. Fallback Strategy
9. Architecture Diagrams
10. Installation Guide
11. Audit Fixes Applied (v2.0)
12. Skill Optimization: api-gateway-skill
13. Skill Optimization: browser-use
14. Skill Optimization: deployment-manager
15. Skill Optimization: persistent-memory
16. Skill Optimization: mcp-builder
17. Security and ToS Risk Assessment
18. Final Recommendations

1. Executive Summary

This guide provides a comprehensive walkthrough for setting up a custom AI development stack on Ubuntu 24.04 LTS, designed specifically for systems with 8GB RAM and Intel Core i5-6200U processors. The stack is organized into two fundamentally different approaches that can operate independently or side-by-side: the **Free Unlimited Stack**, which leverages OpenCode, AntigraVity, and account rotation to achieve effectively unlimited AI usage at zero cost (with ToS risk), and the **Paid Limited Stack**, which uses Claude Pro/Max subscriptions with a billing proxy to maximize the value of an existing paid plan.

The guide covers 15 tools across three tiers, providing detailed installation instructions, compatibility analysis, contradiction mapping, fallback strategies, and security assessments. It incorporates all six audit fixes from the Owl-Agent Audit 2, including the critical transition from a 2-hop proxy chain to a single-hop Python billing proxy, health-check middleware to prevent silent billing bypass, consolidated OAuth token storage, unified Docker/bare-metal configuration paths, version-pinned binaries with SHA256 verification, and comprehensive integration tests. Five skill optimizations (api-gateway-skill, browser-use, deployment-manager, persistent-memory, mcp-builder) are analyzed with stacking recommendations and compatibility matrices.

The enhanced installer script (install-ai-stack-v2.sh) automates the entire setup process with a modular menu system, allowing users to choose between Free Unlimited, Paid Limited, or Both stacks. It includes systemd service creation for auto-start, Prometheus metrics endpoints, Redis-based TTL bloom filter for request deduplication, and a comprehensive health-check script that validates every component of the stack.

2. AI Stack Ecosystem Overview

The AI development ecosystem in 2026 is dominated by three categories of tools: coding agents (OpenCode, Claude Code, Codex, Kiro-CLI), smart routers and billing proxies (n9router, Dario, openclaw-billing-proxy, free-claude-code), and orchestration platforms (Emdash, Hermes Agent, OWL). Each category serves a distinct purpose in the stack, and understanding their relationships is critical for building a reliable and cost-effective development environment.

The fundamental tension in any AI stack is between cost and capability. Free tiers offer generous quotas but impose per-account rate limits that can be disruptive during intensive development sessions. Paid subscriptions remove these limits but introduce hard caps (Claude Pro: approximately 100 messages per 5 hours; Claude Max: approximately 225-900 messages per 5 hours depending on plan tier). Billing proxies like Dario and openclaw-billing-proxy exist to maximize the value of paid subscriptions by routing all tool traffic through a single subscription endpoint, but they cannot bypass Anthropic's hard usage caps. Account rotation tools like AG Proxy Manager and n9router exist to extend free tier usage across multiple accounts, but this practice violates the Terms of Service of most providers and carries a risk of account bans.

2.1 Tier Classification

Tools in the AI stack ecosystem can be classified into three tiers based on their cost structure and access model. Tier 1 tools offer genuinely free access through official free tiers and require no subscription or credit card. Tier 2 tools are free but require account management to circumvent per-account limits. Tier 3 tools require a paid subscription and use billing proxies to maximize subscription value. This classification is essential for understanding the true cost and risk profile of each stack configuration.

Tier	Category	Cost	ToS Risk	Examples
1	Official free tier	Zero	Low	OpenCode, Kiro-CLI, Codex, Emdash, Hermes
2	Free + account rotation	Zero	High	Antigravity + AG Proxy, n9router Token Rotate
3	Paid subscription + proxy	\$20-200/mo	Medium-High	Claude Code + Dario, openclaw-billing-proxy

Table 1: AI Stack Tool Tier Classification

3. Tool Deep Dive: Free Unlimited Stack

The Free Unlimited Stack is designed for users who want zero-cost AI assistance for coding, writing, and research. It achieves "unlimited" usage by combining multiple free-tier quotas through account rotation and smart routing. While the individual tools are legitimate and free, the account rotation mechanisms used to extend their quotas typically violate the Terms of Service of the underlying providers, creating a meaningful risk of account suspension.

3.1 OpenCode CLI

OpenCode is a terminal-based AI coding agent developed by SST (sst/opencode on GitHub) with approximately 162,000 GitHub stars. It supports multiple AI providers through a pluggable architecture, allowing users to connect to Anthropic, OpenAI, Google, or free-tier providers without changing their workflow. The tool operates entirely in the terminal and does not require a separate server process, making it extremely lightweight at approximately 50-100 MB of memory usage. Installation is straightforward via the official installer script or through npx. OpenCode is fully open source and carries no Terms of Service risk when used with official free tiers. The primary limitation is that free tier quotas are per-account, which is where the account rotation tools become relevant.

3.2 Antigravity IDE

Antigravity is Google's official AI-powered IDE, launched in late 2025 as an Electron-based fork of VS Code with deep integration into Google's AI models. It offers a free trial period for all users, after which a Google AI Pro subscription at \$19.99 per month is required for extended use. The IDE itself is

memory-intensive at approximately 300-500 MB due to its Electron foundation, but it provides access to Google's most capable models including Gemini Pro and Claude models through Google's partnerships. When used legitimately within the free trial or paid subscription, Antigravity is a safe and powerful option. The risk arises when third-party tools like AG Proxy Manager and `opencode-antigravity-auth` are used to rotate between multiple free Google accounts to bypass the trial period limitations.

3.3 `opencode-antigravity-auth`

This npm plugin bridges OpenCode with Antigravity's model quotas, allowing OpenCode to use Antigravity's free model tier without launching the full IDE. The plugin handles OAuth2 authentication with Google accounts and routes OpenCode's API calls through Antigravity's endpoints. It is community-maintained (by NoeFabris) and has had security vulnerabilities in versions 1.1.0 through 1.4.6, specifically incorrect default permissions that were documented by Aikido Intel. Users should verify they are running version 1.5.0 or later. The plugin is extremely lightweight at approximately 10-20 MB and requires no separate server process. However, using this plugin to access Antigravity's free tier from OpenCode likely violates Google's Terms of Service, as it circumvents the intended usage model of the Antigravity IDE.

3.4 AG Proxy Manager

AG Proxy Manager (github.com/Ethan-W20/antigravity-proxy-tools) is a Node.js-based account pool manager specifically designed for Antigravity. It aggregates quotas from multiple free Google accounts into a single pool, enabling continuous AI usage by automatically rotating between accounts when one hits its rate limit. The tool includes a "business edition" with additional features. At approximately 30-50 MB of memory, it is lightweight enough for an 8GB system. However, the tool is explicitly designed to bypass 403 authentication errors and use multiple accounts' quotas simultaneously, which almost certainly violates Google Antigravity's Terms of Service. Users should be aware that Google may suspend all accounts in the pool if the rotation pattern is detected, potentially losing access to associated Google services.

3.5 `n9router` (Smart Router)

`n9router` is a Node.js-based smart routing gateway that serves as a universal adapter between AI client applications and multiple AI providers. It features a web dashboard (accessible at <http://localhost:20128>) where users can configure provider accounts, set routing policies, and monitor usage across all connected providers. The tool supports over 40 providers and includes an RTK Token Saver feature that claims to reduce token usage by 20-40% through response caching and deduplication. At approximately 50-80 MB of memory, `n9router` is a moderate resource consumer. It is the recommended replacement for the full Owl-Agent proxy defense stack, as it provides similar functionality (multi-provider routing, automatic failover, request deduplication) through a more user-friendly interface. The ToS risk depends on how it is configured: using it with official free tiers is low-risk, but the "Token Rotate" feature that rotates OAuth tokens across accounts is a clear Terms of Service violation for most providers.

4. Tool Deep Dive: Paid Limited Stack

The Paid Limited Stack is designed for professionals who already subscribe to Claude Pro (\$20/month) or Claude Max (\$100-200/month) and want to maximize the value of their subscription across multiple tools and applications. The core concept is simple but frequently misunderstood: billing proxies do not create free or unlimited usage. They are sophisticated bridges that route tool traffic through an existing subscription, making that subscription accessible to a wider range of applications. When you hit Claude's hard usage cap (approximately 100 messages per 5 hours for Pro, 225-900 for Max), you must wait for the window to reset or switch to pay-as-you-go API credits.

4.1 Claude Code

Claude Code is Anthropic's official CLI coding agent, available as an npm package (@anthropic-ai/claude-code) with approximately 81,600 GitHub stars. It is the most capable coding agent in the ecosystem, with deep understanding of codebases, multi-file editing capabilities, and integration with version control systems. Memory usage is moderate at approximately 100-200 MB. Authentication requires either a Claude Pro/Max subscription (via "claude auth login") or an Anthropic API key for pay-per-token usage. When used as intended, Claude Code carries no Terms of Service risk. The tool is actively maintained with multiple releases per week, and its source code is partially available on GitHub. It serves as the authentication anchor for all billing proxies in the Paid Limited Stack, as these proxies extract the session token from Claude Code's authentication to route traffic through the subscription.

4.2 Dario Billing Proxy

Dario (@askalf/dario) is a purpose-built billing proxy that intercepts API calls from any OpenAI-compatible or Anthropic-compatible tool and routes them through an active Claude Code session. The proxy works by extracting the billing headers that Claude Code injects into its API requests and replicating them for all proxied traffic, making Anthropic's billing classifier treat all requests as if they originated from a Claude Code session. This effectively allows any tool that supports custom API endpoints to use your Claude subscription at no additional per-token cost. The proxy is lightweight at approximately 30-50 MB and runs as a local server that can be configured to auto-start via systemd. The significant drawback is that this billing header injection almost certainly violates Anthropic's Terms of Service, and community discussions confirm that Anthropic has taken action against accounts that use such proxies. Users should weigh the cost savings against the risk of losing their Claude subscription entirely.

4.3 openclaw-billing-proxy

openclaw-billing-proxy (github.com/zacdcook/openclaw-billing-proxy) is similar to Dario but specifically designed for the OpenClaw coding agent. It was created in response to Anthropic's policy change in early April 2026 that introduced "Extra Usage" billing for API traffic that exceeded subscription quotas. The proxy redirects OpenClaw's API calls back through the Claude subscription

billing path, avoiding the more expensive per-token Extra Usage charges. At approximately 20-40 MB, it is the lightest billing proxy option. Security researchers have flagged this tool for requesting full OpenClaw configuration access, which includes sensitive tokens. The ToS risk is equivalent to Dario: billing header injection to circumvent Anthropic's pricing model is almost certainly a violation of the Terms of Service.

4.4 Python Billing Proxy (Audit-Fixed, Single-Hop)

The Python billing proxy included in the v2.0 installer is an audit-fixed replacement for the original 2-hop proxy chain (Python:60000 then Node:4623 then upstream). The original architecture added 40-80ms of unnecessary latency per request and introduced a critical failure mode: if the Node.js middle hop crashed silently, the Python proxy would continue accepting requests but never forward them to the upstream provider, resulting in a silent billing bypass where requests were accepted but never processed. The v2.0 proxy eliminates the Node.js middle hop entirely, routing directly from Python:60000 to the upstream provider. It includes health-check middleware that returns 503 when the circuit breaker is open, preventing the silent bypass. Prometheus metrics are exposed on /metrics for monitoring, and a Redis-based TTL bloom filter provides request deduplication. The consolidated OAuth token is loaded from a single secret file with 0600 permissions, reducing the attack surface compared to the previous dual-storage approach.

5. Tool Deep Dive: Hybrid and Optional Tools

Several tools in the ecosystem work with both the Free Unlimited and Paid Limited stacks, providing cross-stack value regardless of which approach you choose. These tools are installed by the "Hybrid Tools" option in the installer and can be added to either stack configuration.

5.1 Kiro-CLI

Kiro-CLI is Amazon/AWS's agentic IDE and command-line tool, available for free during its preview period. The CLI component is lightweight at approximately 50 MB, while the full Electron IDE requires 300-500 MB. Kiro integrates with AWS Bedrock for model access, which requires an AWS account but offers free tier quotas. In the v2.0 installer, Kiro-CLI is version-pinned (currently v2.3.1) with SHA256 verification to prevent supply chain attacks, addressing a critical audit finding. The primary free access method is through a 9Router/n9router gateway connection, which routes to the best available free endpoint. This is a lower-risk approach than direct account rotation, as it uses communal routing rather than individual account manipulation.

5.2 Codex CLI

Codex is OpenAI's official open-source CLI coding agent, available as @openai/codex on npm with an Apache 2.0 license. It is actively being migrated to a native Rust binary for improved performance and reduced memory footprint. The current Node.js version uses approximately 50-100 MB. Codex requires

an OpenAI API key for operation, which operates on a pay-per-token basis. There is no subscription that provides unlimited usage for Codex; the cost scales directly with usage. For budget-conscious users, Codex can be routed through n9router's free-tier providers, though this configuration is unofficial and may not provide consistent results.

5.3 Emdash (ADE)

Emdash is a Y Combinator W26-backed Agentic Development Environment that orchestrates multiple coding agents in parallel. It provides a desktop dashboard where users can launch and monitor up to 20+ agents (Claude Code, Codex, OpenCode, etc.) with each agent isolated in its own git worktree to prevent conflicts. Installation is via pip (emdash-ai) or a desktop AppImage download. Memory usage is approximately 100-200 MB for the orchestrator process, with each managed agent consuming its own resources. Emdash is the recommended tool for managing multiple agents simultaneously, as it handles git worktree creation, agent lifecycle management, and output aggregation. It carries no ToS risk as it is purely an orchestrator that delegates to agents you already have configured.

5.4 Hermes Agent

Hermes Agent is NousResearch's self-improving AI agent framework, designed for creating custom AI assistants that can learn and adapt over time. It supports local models via Unsloth and Ollama, making it the best option for users who want to run AI models entirely on their own hardware without any cloud API dependencies. Installation is via pip (camel-ai) or a one-line curl installer. Memory usage varies widely depending on the model: running a 7B parameter model locally requires approximately 4-6 GB, which is feasible on an 8GB system with careful memory management. The framework is open source under Apache 2.0 and carries no ToS risk when used with local models.

5.5 OWL Framework (CAMEL-AI)

OWL (Optimized Workforce Learning) by CAMEL-AI is a multi-agent collaboration framework that enables multiple AI agents to work together on complex tasks. It is the legitimate counterpart to the fictional "Owl-Agent" referenced in earlier discussions. OWL provides agent role definition, task decomposition, inter-agent communication, and result aggregation. Installation is via pip (camel-ai) and requires Python 3.11 or later. The framework uses approximately 200-400 MB including the Python runtime and agent processes. It is fully open source and carries no ToS risk. In the context of the AI stack, OWL can be used to orchestrate complex multi-step workflows that require multiple AI providers or models, such as using one agent for code generation and another for code review.

6. Compatibility Matrix

The following matrix shows the compatibility between each pair of tools in the ecosystem. "HIGH" indicates native or recommended integration, "MEDIUM" indicates possible with configuration, "LOW" indicates limited or tangential interaction, and "CONFLICT" indicates tools that should not be used

together. Understanding these relationships is essential for building a stable and functional stack.

Tool	n9router	Dario	OpenCode	Claude Code	Emdash	Billing Proxy
n9router	N/A	MEDIUM	HIGH	HIGH	HIGH	CONFLICT
Dario	MEDIUM	N/A	HIGH	HIGH	HIGH	CONFLICT
OpenCode	HIGH	HIGH	N/A	LOW	HIGH	HIGH
Claude Code	HIGH	HIGH	LOW	N/A	HIGH	HIGH
Emdash	HIGH	HIGH	HIGH	HIGH	N/A	HIGH
Billing Proxy	CONFLICT	CONFLICT	HIGH	HIGH	HIGH	N/A

Table 2: Core Tool Compatibility Matrix

The most important conflict to understand is between n9router/Dario and the Python billing proxy. These tools serve overlapping functions (routing API traffic through subscription endpoints) and should not be run simultaneously pointing to the same upstream provider. Choose one routing layer: either n9router (for multi-provider routing with a dashboard) or the Python billing proxy (for single-provider routing with health checks and Prometheus metrics). Running both creates routing loops and unpredictable behavior.

7. Contradictions and Conflicts

Beyond the simple compatibility matrix, several deeper contradictions exist in the AI stack ecosystem that can cause subtle failures, performance degradation, or security vulnerabilities if not understood and addressed.

7.1 n9router vs. free-claude-code vs. Billing Proxy

These three tools represent fundamentally different approaches to the same problem: making AI models accessible. free-claude-code is a one-to-one backend redirector that routes Claude Code requests to cheaper or free providers like NVIDIA NIM. n9router is a sophisticated universal gateway that manages multiple providers with automatic failover and token rotation. The Python billing proxy routes all traffic through a single subscription endpoint. Running more than one of these simultaneously creates routing conflicts where requests may be processed by multiple proxies in sequence, adding latency and potentially corrupting billing headers. The recommended approach is to choose one routing layer based on your needs: n9router for multi-provider flexibility, billing proxy for single-subscription reliability, or free-claude-code for simple cost reduction on Claude Code specifically.

7.2 Port Conflicts

Multiple tools in the stack run local HTTP servers, and their default ports can conflict if not carefully managed. The following table shows the default port assignments and recommended configurations. The v2.0 installer addresses this by explicitly assigning non-conflicting ports in the unified configuration file.

Component	Default Port	Configured Port (v2.0)	Purpose
Python Billing Proxy	60000	60000	Single-hop billing injection
Health Check (was Node:4623)	4623	4623	Proxy health monitoring
n9router Dashboard	20128	20128	Smart router web UI
openclaw-billing-proxy	4100	4100	OpenClaw billing proxy
Prometheus Metrics	9090	9090	Billing proxy /metrics
free-clause-code	8082	8082	Free-tier redirector

Table 3: Port Allocation Map

7.3 OAuth Token Dual Storage

A critical contradiction in the original stack design was the storage of OAuth tokens in two separate locations: one in the application configuration file (`~/.config/openhuman-proxy/config.env`) and another in the proxy's internal state. This dual storage doubled the attack surface and created consistency risks where one copy could be updated while the other remained stale. The v2.0 audit fix consolidates all secrets into a single file at `~/.ai-stack/secrets/proxy-secrets.env` with 0600 file permissions (owner read/write only). All tools read from this single source of truth, eliminating the consistency risk and reducing the attack surface by half.

7.4 Docker vs. Bare-Metal Config Path Divergence

When running tools in Docker containers, configuration paths differ from bare-metal installations because container volumes are mounted at different locations. This creates operational complexity where the same tool requires different configuration files depending on the deployment method. The v2.0 audit fix introduces a unified configuration system that uses environment variables (`AI_STACK_DATA_DIR`, `AI_STACK_LOG_DIR`, `AI_STACK_RUN_DIR`) to define all paths. Docker deployments set these variables to container volume mount points, while bare-metal installations use the default `$HOME/.ai-stack/` structure. This eliminates the path divergence entirely and allows the same configuration templates to work in both environments.

8. Fallback Strategy

A resilient AI stack degrades gracefully when components fail. The following four-tier fallback strategy ensures that development work can continue even when the primary AI provider or routing layer is unavailable. Each tier represents a lower level of service quality, from full-featured AI assistance to basic local-only capabilities.

8.1 Primary Path: Full Stack

The primary path routes all AI requests through the full stack: IDE/Agent through Emdash (orchestration) through n9router (smart routing) to the AI provider. This provides the best experience with automatic provider selection, failover, and billing management. All features are available including multi-model workflows, semantic caching, and usage analytics.

8.2 First Fallback: Direct Provider

If n9router fails or becomes unavailable, the IDE/Agent should be configured to bypass it and connect directly to the primary AI provider (e.g., Anthropic for Claude, OpenAI for GPT). This is accomplished by setting `ANTHROPIC_BASE_URL` or `OPENAI_BASE_URL` directly to the provider's API endpoint. The trade-off is loss of smart routing, automatic failover, and the RTK Token Saver, but basic AI functionality is preserved. The environment configuration files (`free-unlimited.env` and `paid-limited.env`) include commented-out fallback URL lines for quick activation.

8.3 Second Fallback: Alternative Provider

If the primary provider is also unavailable (e.g., Anthropic outage), the stack can fall back to an alternative provider. For Free Stack users, this means switching from Antigravity to OpenRouter's free models or a local Ollama instance. For Paid Stack users, this means switching from Claude to GPT-4 or Gemini. This fallback requires changing the provider configuration in the IDE/Agent, which can be automated with a shell alias that sources a different environment file.

8.4 Final Fallback: Local Models

As a last resort, all cloud providers can be bypassed by using local AI models through Ollama or Hermes Agent with Unsloth. This requires sufficient RAM and GPU resources to run models locally. On an 8GB system, a 7B parameter model (like Llama 3.1 8B or Mistral 7B) can run with 4-bit quantization using approximately 4-6 GB of RAM. While the quality of local models is significantly lower than cloud-based models, this tier ensures that basic AI assistance is always available regardless of network conditions or provider outages.

9. Architecture Diagrams

The following text-based architecture diagrams illustrate the data flow for each stack configuration. These diagrams show how requests flow from the developer's IDE through the various proxy and

routing layers to the upstream AI providers.

9.1 Free Unlimited Stack Flow

Developer --> OpenCode/Antigravity --> n9router (:20128) --> Provider Pool

|--> Google Antigravity (Account 1)

|--> Google Antigravity (Account 2)

|--> OpenRouter (Free models)

|--> NVIDIA NIM (Free tier)

9.2 Paid Limited Stack Flow (v2.0 Single-Hop)

Developer --> Any AI Tool --> Python Billing Proxy (:60000) --> Anthropic API

|

+-- /health --> Circuit breaker status

+-- /metrics --> Prometheus endpoint

+-- Redis --> TTL bloom dedup

9.3 Legacy 2-Hop Flow (Deprecated)

Developer --> Python Proxy (:60000) --> Node.js Proxy (:4623) --> Upstream

[DEPRECATED: 2-hop adds 40-80ms latency and silent bypass risk]

10. Installation Guide

The enhanced installer script (install-ai-stack-v2.sh) automates the setup of the entire AI stack on Ubuntu 24.04 LTS. It provides a modular menu system that allows you to install only the components you need, with separate configuration files for the Free Unlimited and Paid Limited stacks.

10.1 Prerequisites

Before running the installer, ensure your system meets the following requirements: Ubuntu 24.04 LTS (or compatible Debian-based distribution), at least 8 GB of RAM, at least 20 GB of free disk space, and a stable internet connection. The installer will check for and install system dependencies (curl, wget, git, build-essential, python3, nodejs, npm) automatically. No pre-configuration is required; the installer handles everything from dependency installation to systemd service creation.

10.2 Quick Start

1. Download the installer:

wget https://your-server/install-ai-stack-v2.sh

2. Make it executable:

```
chmod +x install-ai-stack-v2.sh
```

- 3. Run the installer:
`./install-ai-stack-v2.sh`
- 4. Choose your stack (1=Free, 2=Paid, 3=Both)
- 5. Activate the environment:
`source ~/.ai-stack/free-unlimited.env # or paid-limited.env`
- 6. Verify with health check:
`~/ai-stack-health.sh`
- 7. Run integration tests:
`~/ai-stack/install/tests/run_tests.sh`

10.3 Post-Installation Configuration

After installation, several manual steps are required to complete the setup. For the Free Unlimited Stack, you need to configure the AG Proxy Manager with your Google accounts by editing the accounts.json file. For the Paid Limited Stack, you need to run "claude auth login" to authenticate with your Claude subscription, then edit the proxy-secrets.env file to add any additional API keys. The billing proxy systemd service should be enabled with "systemctl --user enable ai-billing-proxy" and started with "systemctl --user start ai-billing-proxy". Verify the proxy is running by visiting <http://localhost:60000/health> in your browser, which should return a JSON response with status "healthy".

11. Audit Fixes Applied (v2.0)

The v2.0 installer incorporates all six active findings from the Owl-Agent Audit 2. Each fix addresses a specific security, reliability, or performance issue identified during the audit process. The following table summarizes each finding, its severity, the fix applied, and the verification method.

Severity	Finding	Fix Applied	Verification
HIGH	2-hop proxy latency (40-80ms)	Ported billing to Python, eliminated Node.js hop	/health returns "single-hop"
HIGH	Silent billing bypass on crash	Health-check middleware returns 503 when circuit breaker open	curl /health shows circuit_breaker state

MEDI UM	Dual OAuth token storage	Consolidated to single secret file (0600)	stat -c %a shows 600
MEDI UM	Docker/bare-metal path divergence	Unified paths via AI_STACK_* env vars	Same config works in both modes
MEDI UM	Kiro-CLI URL not version-pinned	Version pin with SHA256 verification	verify_sha256() called before install
LOW	Missing billing proxy health check	Added /health endpoint to diagnostics script	ai-stack-health.sh checks :60000

Table 4: Audit Fix Summary

In addition to the six audit fixes, three new features were added to improve observability and reliability: Prometheus metrics are now exposed on the /metrics endpoint (port 60000), providing real-time request counts, latency histograms, active connection gauges, and circuit breaker state. A Redis-based TTL bloom filter provides request deduplication, reducing upstream API costs by serving duplicate requests from cache with a configurable TTL. Finally, a comprehensive integration test suite validates the entire stack, including health checks, Prometheus endpoints, Redis connectivity, secret file permissions, circuit breaker state, and single-hop architecture verification.

12. Skill Optimization: api-gateway-skill

The API Gateway Skill operates at the application layer, receiving explicit API requests and routing them to the optimal upstream provider based on configurable routing policies. Its core strength is the ability to present a single OpenAI-compatible endpoint to clients while intelligently selecting among multiple providers behind the scenes. The skill supports model-based routing (route French-language requests to Claude for superior French performance), cost optimization (route to the cheapest provider for the requested model), latency optimization (route to the fastest provider), and geographic routing (route to the nearest provider).

The optimization for the AI stack context involves integrating the API Gateway with both the Python billing proxy and n9router. In the recommended configuration, the API Gateway sits between the client applications and the routing layer, providing application-level features (semantic caching, request queuing, rate limiting) while delegating provider selection and account management to n9router or the billing proxy. This separation of concerns allows the API Gateway to focus on its core competency (request processing) while the routing layer handles provider selection. The key stacking combination is API Gateway + Combined Proxy Billing, which provides complete AI API management: routing, billing, rate limiting, and caching in a single integrated system.

12.1 Optimization Recommendations

- Add health-check middleware to detect upstream provider failures before they impact clients
- Integrate Prometheus metrics for routing decision visibility and cache hit rate monitoring

- Enable Redis-backed semantic caching to reduce upstream API costs by 15-30%
- Configure canary routing for new provider integrations before full traffic cutover
- Add OpenTelemetry tracing for end-to-end request visibility across the proxy chain

13. Skill Optimization: browser-use

Browser-Use provides natural-language browser automation that is critical for automating OAuth2 consent flows during proxy installation. Its primary value in the AI stack is eliminating manual browser interaction during setup, particularly for the Openhuman Owl Install Proxy which requires an OAuth2 PKCE flow for each developer machine. Browser-Use can automate the entire flow: open the consent URL, wait for SSO authentication, capture the authorization code from the redirect, and pass it back to the installation script.

The key optimization is to expose Browser-Use as an MCP tool using the MCP Builder skill, allowing AI agents to trigger browser automation directly from their workflow. This enables scenarios like "Claude, please set up the proxy on this machine" where the AI agent uses Browser-Use to handle the interactive OAuth2 flow autonomously (with human-in-the-loop for password entry). The session checkpoint and resume capability is particularly valuable for flaky OAuth2 flows where network timeouts or page crashes can interrupt the authentication process, as it allows resuming from the last successful checkpoint rather than restarting from scratch.

13.1 Optimization Recommendations

- Expose as MCP tool for AI-agent-driven OAuth2 automation
- Add persistent session state via persistent-memory for cross-run cookie persistence
- Implement visual assertion for post-deployment UI verification
- Add screenshot-based debugging for OAuth2 flow failures
- Configure adaptive wait strategies for dynamic JavaScript-heavy OAuth2 pages

14. Skill Optimization: deployment-manager

The Deployment Manager serves as the infrastructure backbone for the entire AI stack, automating the building, testing, deploying, monitoring, and rolling back of all stack components. Its canary deployment support is particularly valuable for the billing proxy, where a misconfigured deployment could silently route traffic incorrectly or lose billing headers. The Deployment Manager can deploy a new billing proxy version to a canary instance, route 5% of traffic to it, monitor for billing accuracy and error rates, and either promote to full deployment or rollback based on the canary metrics.

The optimization focuses on creating deployment pipelines for each stack component with appropriate health checks and rollback triggers. The billing proxy deployment should verify that the /health endpoint

returns "healthy" and that Prometheus metrics show no spike in error rates before promoting. The n9router deployment should verify dashboard accessibility and provider connectivity. The API Gateway deployment should verify routing correctness with sample requests before accepting live traffic. The deployment dry-run feature allows validating the entire pipeline without actually deploying, catching configuration errors before they impact production.

14.1 Optimization Recommendations

- Create component-specific deployment pipelines with health check gates
- Add canary deployment for billing proxy with billing accuracy verification
- Implement automatic rollback triggered by Prometheus alert rules
- Add deployment dry-run validation before production cutover
- Integrate with persistent-memory for deployment state persistence across restarts

15. Skill Optimization: persistent-memory

Persistent Memory is the most universally stackable skill, serving as the substrate that makes all other skills reliable across process restarts, shell session terminations, and machine reboots. In the AI stack context, it is the glue that holds everything together: OAuth2 tokens and refresh tokens for proxy re-authentication, quota thresholds and current spend for billing enforcement, routing policies and cache state for gateway recovery, and connection pool state for relay reconnection. Without Persistent Memory, every process restart requires manual reconfiguration, making the stack fragile and operationally expensive to maintain.

The optimization involves consolidating the various ad-hoc state storage mechanisms (flat files, environment variables, Redis key-value pairs) into a unified Persistent Memory namespace hierarchy. Each skill gets its own namespace (e.g., "billing.quota", "gateway.routing", "proxy.tokens") with appropriate TTL settings (tokens expire and auto-clean, routing policies persist indefinitely). The optimistic concurrency control feature is critical for the billing service, where multiple proxy instances may try to update the same team's spend counter simultaneously without losing updates.

15.1 Optimization Recommendations

- Consolidate all state storage into namespaced Persistent Memory
- Set TTL on OAuth2 tokens matching provider expiration policies
- Enable optimistic concurrency control for billing counters
- Configure state snapshots before each deployment for instant rollback
- Add change notification webhooks for token expiration alerts

16. Skill Optimization: mcp-builder

The MCP Builder creates Model Context Protocol servers that expose AI stack capabilities as AI-agent-consumable tools. In the AI stack context, MCP servers are the API layer that makes stack management accessible to AI agents without requiring the human operator to switch to a separate dashboard or CLI tool. For example, an MCP tool called "get_spending" allows Claude to proactively check the team's AI spending during a coding session and suggest cost optimizations.

The key optimization is to build MCP servers for the three most valuable stack capabilities: billing management (get_spending, check_quota, suggest_optimization), routing control (get_routing_policy, update_routing_rule, test_routing), and health monitoring (get_stack_health, get_provider_status, get_circuit_breaker_state). Each MCP tool should use the annotation system to mark safety characteristics: get_spending as readOnly (safe to call anytime), update_routing_rule as destructive (requires human confirmation), and get_stack_health as readOnly (safe for autonomous monitoring). The evaluation mode should be used to validate that all tools have descriptions, valid schemas, and appropriate annotations before deployment.

16.1 Optimization Recommendations

- Build MCP server for billing management (get_spending, check_quota, suggest_optimization)
- Build MCP server for routing control with destructiveHint annotations
- Build MCP server for health monitoring with readOnlyHint annotations
- Use evaluation mode to validate all MCP servers against specification
- Add resource subscriptions for real-time billing threshold alerts

17. Security and ToS Risk Assessment

The AI stack ecosystem contains several tools that carry significant security or Terms of Service risks. Understanding these risks is essential for making informed decisions about which tools to include in your stack. The following table provides a comprehensive risk assessment for each tool, categorized by risk type and severity.

Tool	Risk Category	Severity	Description	Mitigation
AG Proxy Manager	ToS Violation	HIGH	Bypasses Antigravity auth, uses multiple accounts	Use only with legitimate paid accounts
opencode-antigravity-auth	ToS Violation	MEDIUM	Routes OpenCode through Antigravity free tier	Limit to personal single-account use
Dario	ToS Violation	HIGH	Injects billing headers to use Claude subscription	Accept risk or use API keys directly
openclaw-billing-proxy	ToS Violation	HIGH	Redirects billing through Claude subscription	Accept risk or use API keys directly

free-claude-cod e	Supply Chain	HIGH	Known malicious npm packages impersonating it	Use only verified GitHub repos
n9router Token Rotate	ToS Violation	MEDI UM	Rotates OAuth tokens across accounts	Use only provider rotation, not token rotation
Python Billing Proxy	ToS Violation	MEDI UM	Same billing header injection as Dario	Use only with your own subscription

Table 5: Security and ToS Risk Assessment

The supply chain attack surface is particularly concerning for the "free Claude Code" ecosystem. Multiple security firms (SafeDep, Snyk, Microsoft) have documented malicious npm packages that impersonate free Claude Code tools, shipping hidden ELF binaries and hooking into Claude Code's SessionStart event to steal API keys and credentials from the ~/.claude/ directory. Users should install tools only from their official GitHub repositories and verify SHA256 checksums when available. The v2.0 installer includes SHA256 verification for all version-pinned binaries as a supply chain defense measure.

18. Final Recommendations

Based on the comprehensive analysis of all 15 tools, their compatibility relationships, contradiction patterns, and risk profiles, the following tiered recommendation provides a clear path forward for different user scenarios. The key principle is to start simple and add complexity only when needed, prioritizing reliability and safety over feature completeness.

18.1 Tier 1: The Minimalist (Recommended Starting Point)

Start with only OpenCode + opencode-antigravity-auth (single account) + Kiro-CLI. This provides genuinely free AI assistance with minimal ToS risk, using official free tiers within their intended limits. Memory footprint is minimal (under 200 MB total), and the setup is simple enough to debug when issues arise. This tier is ideal for students, hobbyists, and anyone evaluating whether AI-assisted coding is valuable for their workflow before committing to a paid plan.

18.2 Tier 2: The Router (Adding Flexibility)

Add n9router as a smart routing layer between your tools and AI providers. This provides multi-provider failover, automatic provider selection, and the RTK Token Saver for reduced token usage. Configure n9router to use official free tier endpoints (avoid the Token Rotate feature to minimize ToS risk). This tier is ideal for developers who find the free tier of a single provider insufficient and want access to multiple providers without manual switching. The memory overhead is moderate (approximately 250 MB additional), well within the budget of an 8GB system.

18.3 Tier 3: The Professional (Maximum Capability)

Add a Claude Pro subscription (\$20/month) with the Python billing proxy for reliable, high-quality AI assistance across all tools. This tier uses the billing proxy to make the Claude subscription accessible to OpenCode, Codex, and any other OpenAI-compatible tool. Be aware that the billing proxy's header injection mechanism likely violates Anthropic's ToS, and you should accept this risk knowingly. For organizations that can justify the cost, Claude Max (\$100-200/month) provides significantly higher usage caps. Emdash should be added at this tier for managing multiple agents in parallel with git worktree isolation.

18.4 Architecture Recommendation

The recommended architecture is to skip free-clause-code and openclaw-billing-proxy entirely. n9router handles all routing needs more comprehensively than free-clause-code, and the Python billing proxy (included in the v2.0 installer) provides health checks and Prometheus metrics that openclaw-billing-proxy lacks. The stack should be built incrementally: start with the core (n9router + OpenCode), add orchestration (Emdash) when you need multi-agent workflows, and add the billing proxy when you subscribe to Claude. This incremental approach makes troubleshooting far easier than installing everything at once, as each addition can be validated independently before moving to the next component.

Recommendation	Rationale	Priority
Use n9router as primary router	Multi-provider support, dashboard UI, RTK Token Saver	HIGH
Skip free-clause-code	Redundant with n9router, known supply chain attacks	HIGH
Skip openclaw-billing-proxy	Redundant with Python billing proxy, security concerns	MEDIUM
Add Emdash for orchestration	Multi-agent management with git worktree isolation	MEDIUM
Use Python billing proxy (v2.0)	Health checks, Prometheus, single-hop, audit-fixed	HIGH
Start with Tier 1, increment to Tier 3	Debugging simplicity, risk awareness	HIGH
Keep Owl-Agent defense stack optional	Only needed for extreme privacy/network issues	LOW

Table 6: Final Recommendations Summary